МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«_____»

Кафедра _____

ЛАБОРАТОРНАЯ РАБОТА № 4

по дисциплине «Основы алгоритмизации и программирования»

Анализ алгоритмов последовательного, двоичного и интерполяционного поиска

Вариант 17

Направление:	38.03.05 «Бизнес-информатика»
Курс:	2
Выполнила:	студент группы БИ 2-1
ФИО:	Осипова Анна Алексеевна
Проверил:	_____

2026

1. Цель работы

Изучить алгоритмы поиска элемента в массиве, экспериментально сопоставить их трудоёмкость с теоретическими оценками и определить условия, при которых предварительная сортировка данных оправдана.

2. Задание варианта 17

Параметр	Значение
Размеры массива n	1000; 3162; 10 000; 31 623; 100 000 (5 точек)
Распределение ключей	C — степенное: floor(10^6 * u^3), u ~ U[0;1)
Число запросов k	1500
Тип запросов	успешные
Дополнительное задание	БЗ — точка окупаемости сортировки

Язык реализации: Python 3.10+.

3. Теоретическая часть

3.1. Последовательный поиск

Элементы просматриваются подряд. Упорядоченность не требуется. Среднее число сравнений при успешном поиске: C_ср = (n+1)/2 = Θ(n). При неуспешном поиске — всегда n сравнений.

3.2. Двоичный поиск

Только для упорядоченного массива. После каждого сравнения длина отрезка уменьшается вдвое: C_max = floor(log2 n) + 1 = Θ(log n).

3.3. Интерполяционный поиск

Пробная позиция: mid = lo + (q − A[lo])·(hi − lo) // (A[hi] − A[lo]). При равномерном распределении — O(log log n); при степенном — до O(n).

3.4. Окупаемость сортировки

T_без = k · c1 · n, T_с = c2 · n · log n + k · c3 · log n. Сортировка окупается при k (c2/c1)·log n — порог растёт логарифмически.

3.5. Сводная таблица

Алгоритм	Упорядоч.	Лучший	Средний	Худший	Память
Последовательный	Нет	Θ(1)	Θ(n)	Θ(n)	O(1)
Двоичный	Да	Θ(1)	Θ(log n)	Θ(log n)	O(1)
Интерполяционный	Да	Θ(1)	Θ(log log n)*	Θ(n)	O(1)

* При равномерном распределении ключей.

4. Блок-схемы алгоритмов

4.1. Последовательный поиск

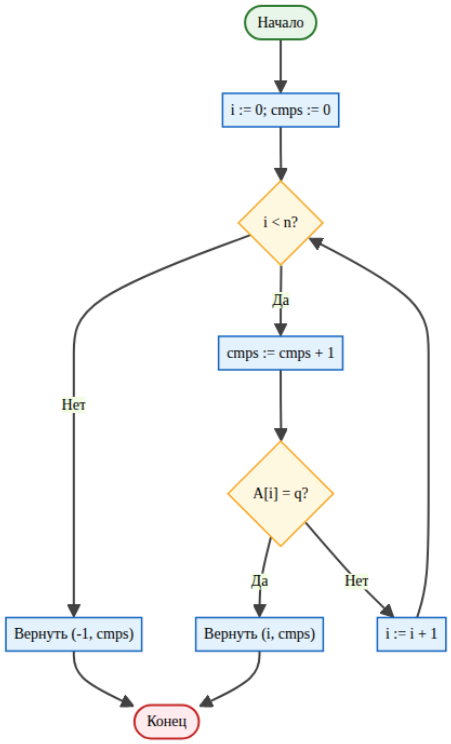


Рисунок 1 — Блок-схема последовательного поиска

4.2. Двоичный поиск

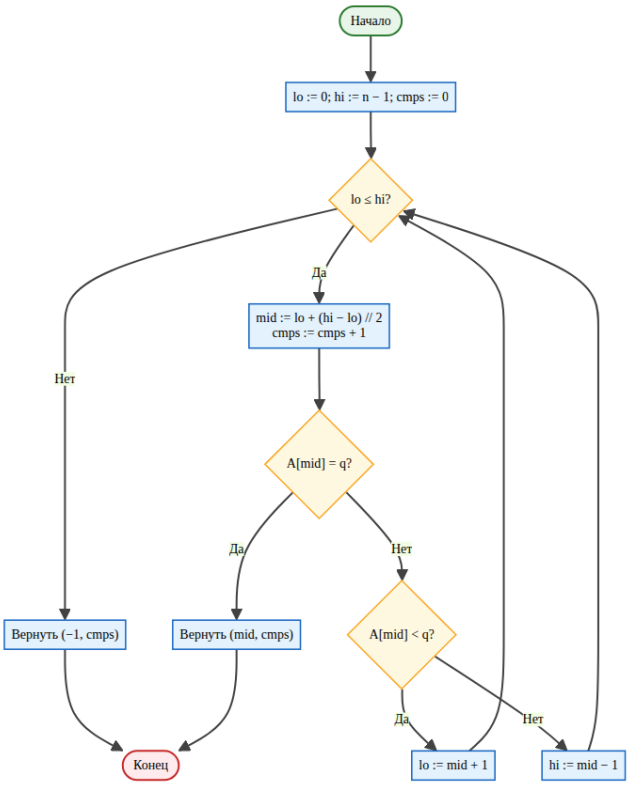


Рисунок 2 — Блок-схема двоичного поиска

4.3. Интерполяционный поиск

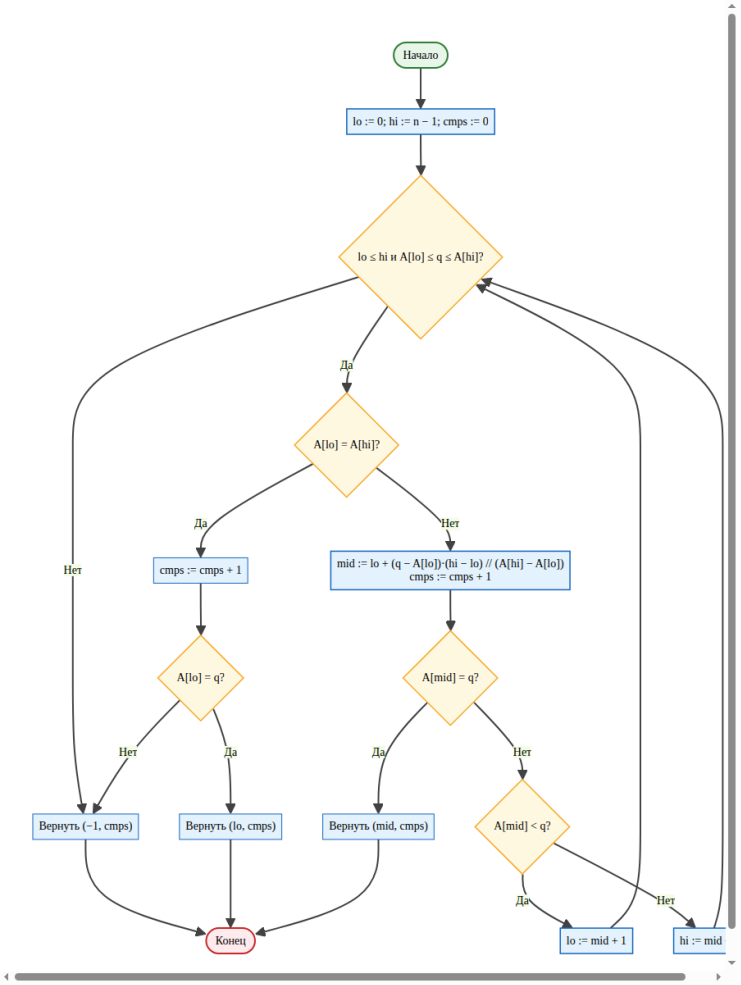


Рисунок 3 — Блок-схема интерполяционного поиска

5. Листинг программы

```
def linear_search(a, q):
    for i, value in enumerate(a):
        if value == q:
            return i, i + 1
    return -1, len(a)
def binary_search(a, q):
    lo, hi, cmps = 0, len(a) - 1, 0
    while lo <= hi:
        mid = lo + (hi - lo) // 2
        cmps += 1
        if a[mid] == q: return mid, cmps
        if a[mid] < q: lo = mid + 1
        else: hi = mid - 1
    return -1, cmps
def interpolation_search(a, q):
    lo, hi, cmps = 0, len(a) - 1, 0
    while lo <= hi and a[lo] <= q <= a[hi]:
        if a[lo] == a[hi]:
            cmps += 1
            return (lo, cmps) if a[lo] == q else (-1, cmps)
        mid = lo + (q - a[lo]) * (hi - lo) // (a[hi] - a[lo])
        mid = max(lo, min(mid, hi))
        cmps += 1
        if a[mid] == q: return mid, cmps
        if a[mid] < q: lo = mid + 1
```

```
        else: hi = mid - 1
    return -1, cmps
def make_power_data(n, seed=42):
    rng = random.Random(seed + n)
    data = [int(1_000_000 * (rng.random() ** 3)) for _ in range(n)]
    data.sort()
    return data
```

6. Характеристики вычислительной системы

Измерения: Linux, Python 3.10+, time.perf_counter(). Среднее время запроса = суммарное время серии / k. Число сравнений усреднено по 1500 успешным запросам.

7. Результаты измерений

7.1. Среднее время одного запроса, мкс

n	Последовательный	Двоичный	Интерполяционный
1 000	12,92	1,03	8,64
3 162	64,93	1,24	12,71
10 000	147,73	1,33	21,01
31 623	559,69	1,69	34,40
100 000	2294,91	1,97	59,71

7.2. Среднее число сравнений на один запрос

n	Послед.	Теория (n+1)/2	Двоичн.	Теория log2 n	Интерп.
1 000	507,5	500,5	9,0	10,0	22,5
3 162	1588,3	1581,5	10,6	11,6	33,9
10 000	5075,3	5000,5	12,2	13,3	49,8
31 623	15873,9	15812,0	13,7	15,0	78,0
100 000	50343,2	50000,5	15,0	16,6	108,0

Интерполяционный поиск на степенном распределении деградирует: число сравнений растёт и уже при n = 1000 превышает двоичный поиск. Формула пробной позиции предполагает равномерность, которой нет при u^3.

7.3. Графики

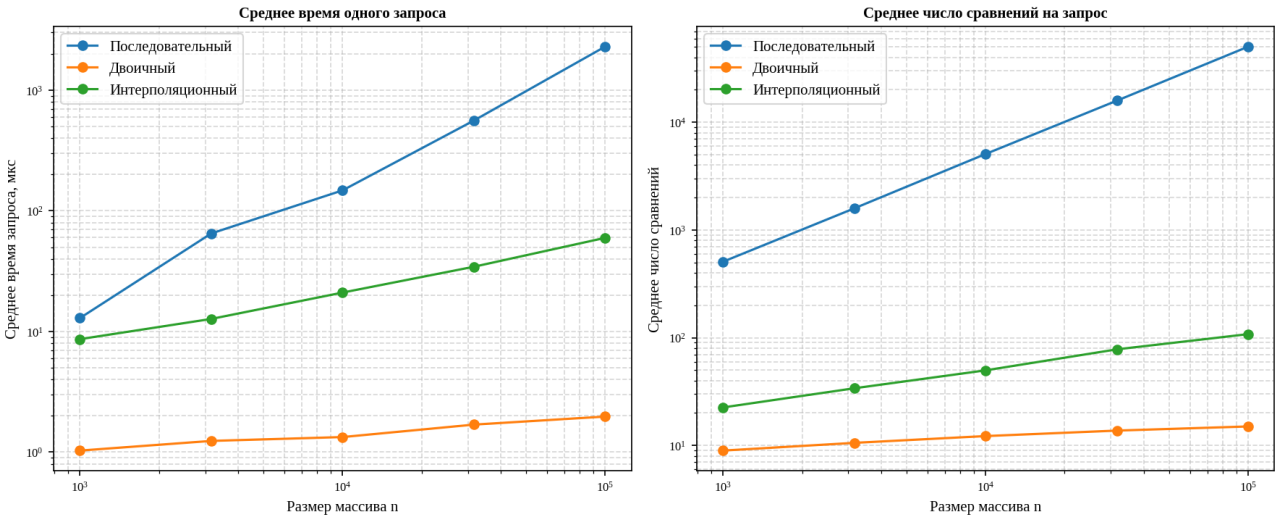


Рисунок 4 — Среднее время и число сравнений (лог. шкалы)

8. Дополнительное задание Б3 — точка окупаемости сортировки

Фиксированный n = 100 000. Стратегии: без сортировки (k · t_lin) и с сортировкой (t_sort + k · t_bin).

Параметр	Значение
t_lin (последовательный)	1482,3 мкс/запрос
t_bin (двоичный)	2,53 мкс/запрос
t_sort (Timsort)	25,3 мс
k* (пересечение)	≈ 17,1

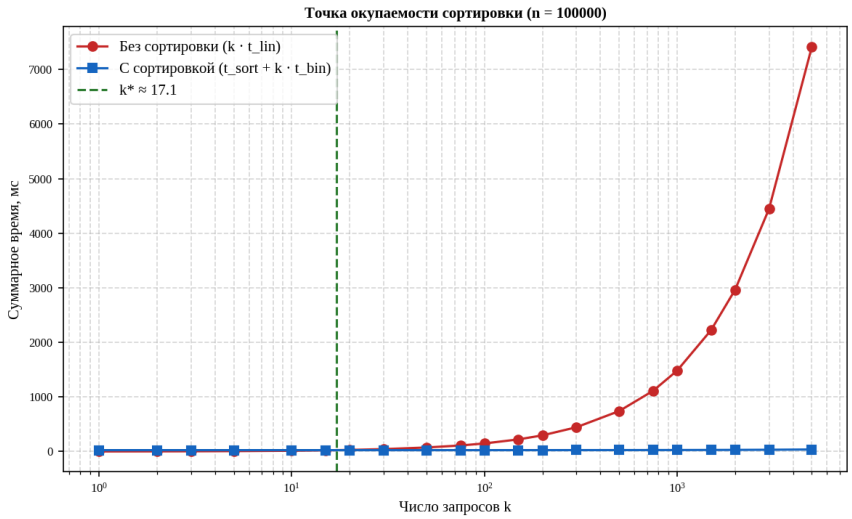


Рисунок 5 — Точка окупаемости сортировки (n = 100 000)

Уже при ~17 запросах предварительная сортировка окупается. При k = 1500 стратегия с сортировкой быстрее примерно в 90 раз. Это объясняет роль индексов в БД.

9. Выводы

1. Эксперимент подтверждает теорию: последовательный поиск — Θ(n), двоичный — Θ(log n).
2. На степенном распределении интерполяционный поиск уступает двоичному и по числу сравнений, и по времени. Вывод о его превосходстве справедлив лишь для равномерных данных.
3. Точка окупаемости сортировки при n = 10^5 составляет k* ≈ 17. Порог логарифмический — предварительная сортировка (или индекс) оправдана уже при небольшом числе запросов.

4. В прикладных системах для многократного поиска по большим данным следует поддерживать упорядоченную структуру, а не выполнять линейный просмотр.

10. Контрольные вопросы

1. Задача поиска. Успешный и неуспешный поиск.

   Найти индекс элемента с заданным ключом или установить отсутствие. Неуспешный поиск обычно требует больше операций.

2. Среднее число сравнений последовательного поиска.

   При равновероятном расположении: C = (n+1)/2.

3. Барьерный элемент.

   В конец массива записывается искомый ключ — цикл гарантированно завершится. Асимптотика Θ(n) не меняется.

4. Почему двоичный поиск требует упорядоченного массива.

   Решение «влево/вправо» основано на упорядоченности; иначе результат некорректен.

5. Число сравнений двоичного поиска при n = 10^6.

   floor(log2 10^6) + 1 ≈ 20.

6. Пробная позиция интерполяционного поиска.

   Линейная интерполяция в предположении равномерного распределения значений.

7. Когда интерполяция деградирует до Θ(n).

   При сильно неравномерном распределении (степенное, кластерное).

8. Условие окупаемости сортировки.

   k (c2/c1)·log n. Порог логарифмический.

9. Индексы в БД.

   Ускоряют выборку, но замедляют вставку (нужно поддерживать индекс).

10. Все вхождения ключа за O(log n + m).

    Двоичным поиском найти первое и последнее вхождение, затем выдать отрезок длины m.

11. Рекомендуемая литература

1. Вирт Н. Алгоритмы и структуры данных. — М.: ДМК Пресс, 2016.
2. Кнут Д. Э. Искусство программирования. Т. 3. — 2-е изд. — М.: Вильямс, 2017.
3. Кормен Т. и др. Алгоритмы: построение и анализ. — 3-е изд. — М.: Вильямс, 2013.
4. Седжвик Р., Уэйн К. Алгоритмы на Java. — 4-е изд. — М.: Вильямс, 2013.
5. Документация Python: модуль bisect — https://docs.python.org/3/library/bisect.html